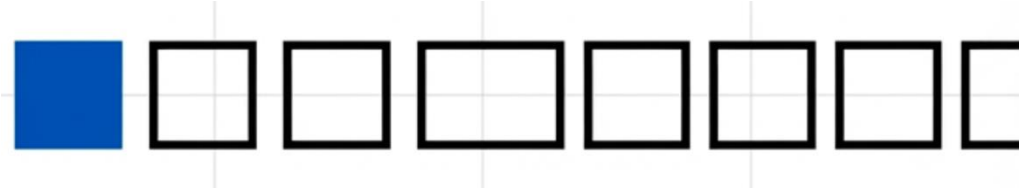
Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Clase std::vector

Fundamentos y gestión de memoria



Objetivos

- Comprender la importancia de la **contigüidad física** en el *Heap*.
- Diferenciar entre la copia profunda (O(n)) y la transferencia de punteros (**Move Constructor**, O(1))
- Evaluar el uso de emplace_back (construcción *in-place*) frente a push_back.
- Analizar la diferencia técnica entre **Size** (lógica) y **Capacity** (física).
- Implementar reserve() para suprimir el costo de las reasignaciones dinámicas y comparar su impacto frente a resize().

Definición Fundamental: El Arreglo Dinámico

¿Qué es?

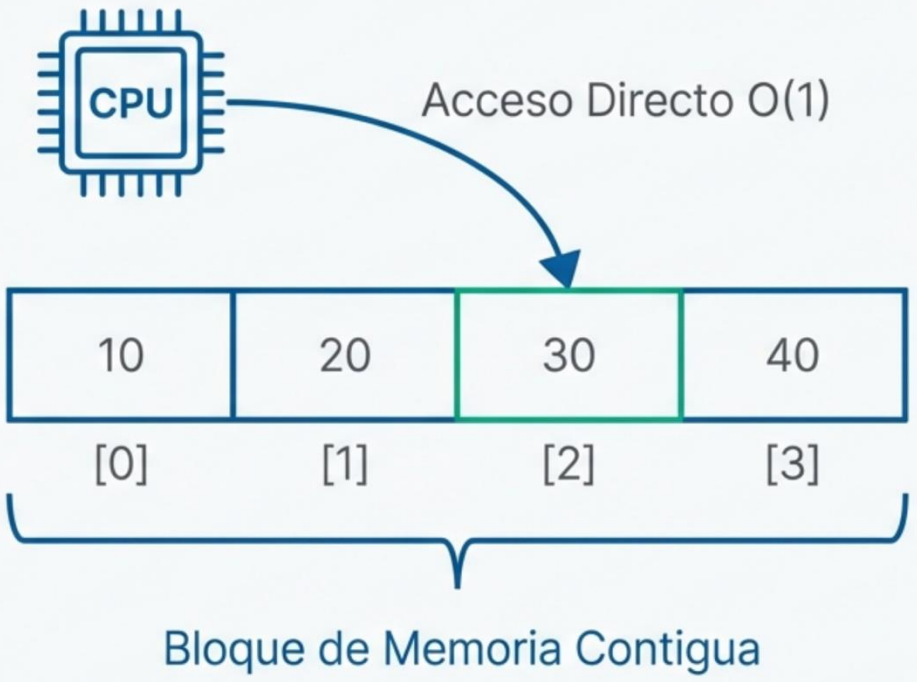
std::vector es un contenedor de secuencia que encapsula arreglos de tamaño dinámico.

La Ventaja Arquitectónica:

A diferencia de las listas, los elementos se almacenan de forma **contigua** en la memoria.

Beneficio Clave:

Acceso aleatorio O(1) y optimización de Localidad de Caché (Cache Locality).



El Problema: Gestión Manual en Arreglos C

La complejidad y el riesgo de redimensionar manualmente.

```
// El 'dolor' de redimensionar manualmente
int* cArray = new int[5];
// ... inicialización ...

// Para crecer, debemos asignar, copiar y borrar
int* newCArray = new int[7];
for (int i = 0; i < 5; ++i) newCArray[i] = cArray[i];
delete[] cArray; // ¡Riesgo si se olvida esta línea!
cArray = newCArray;
```

Propenso a Fugas de Memoria (Memory Leaks) y errores humanos.

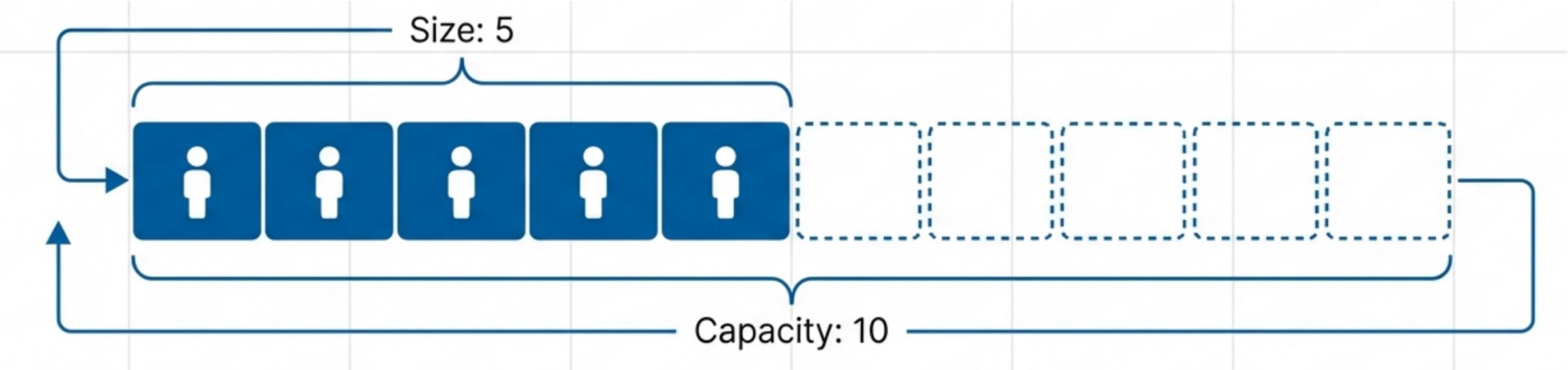
La Solución: Gestión Automática con std::vector

```
// La simplicidad de la STL
std::vector<int> stlVector = {1, 2, 3, 4, 5};

// Redimensionar es una sola línea
stlVector.resize(7);

// No hay 'new' ni 'delete'.
// El destructor limpia la memoria automáticamente.
```

RAII Gestión automática de memoria: Cero fugas de memoria.

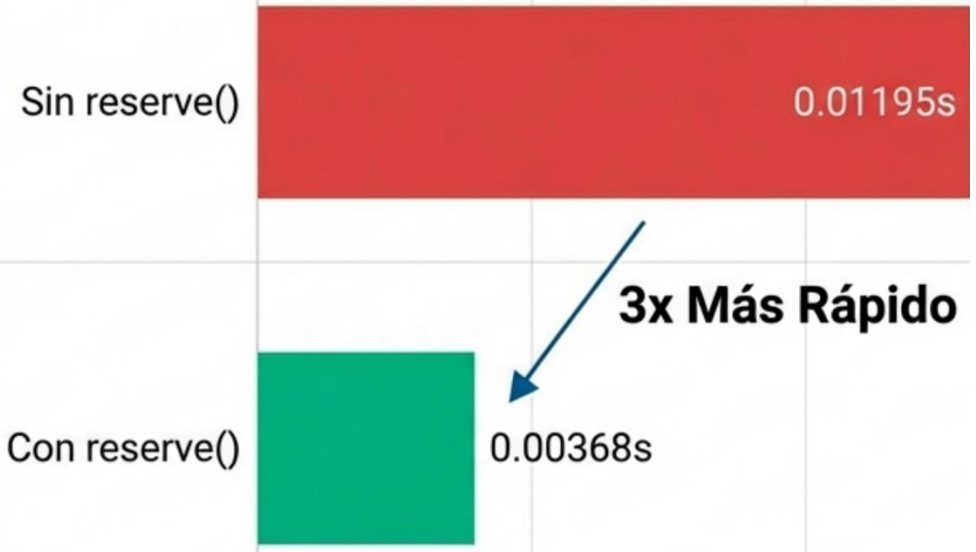
Mecánica Interna: Tamaño vs. Capacidad

Size (Tamaño): Número de elementos que el vector contiene actualmente.
Capacity (Capacidad): Cantidad total de memoria reservada (incluyendo espacio libre).



Optimización de Memoria: reserve()

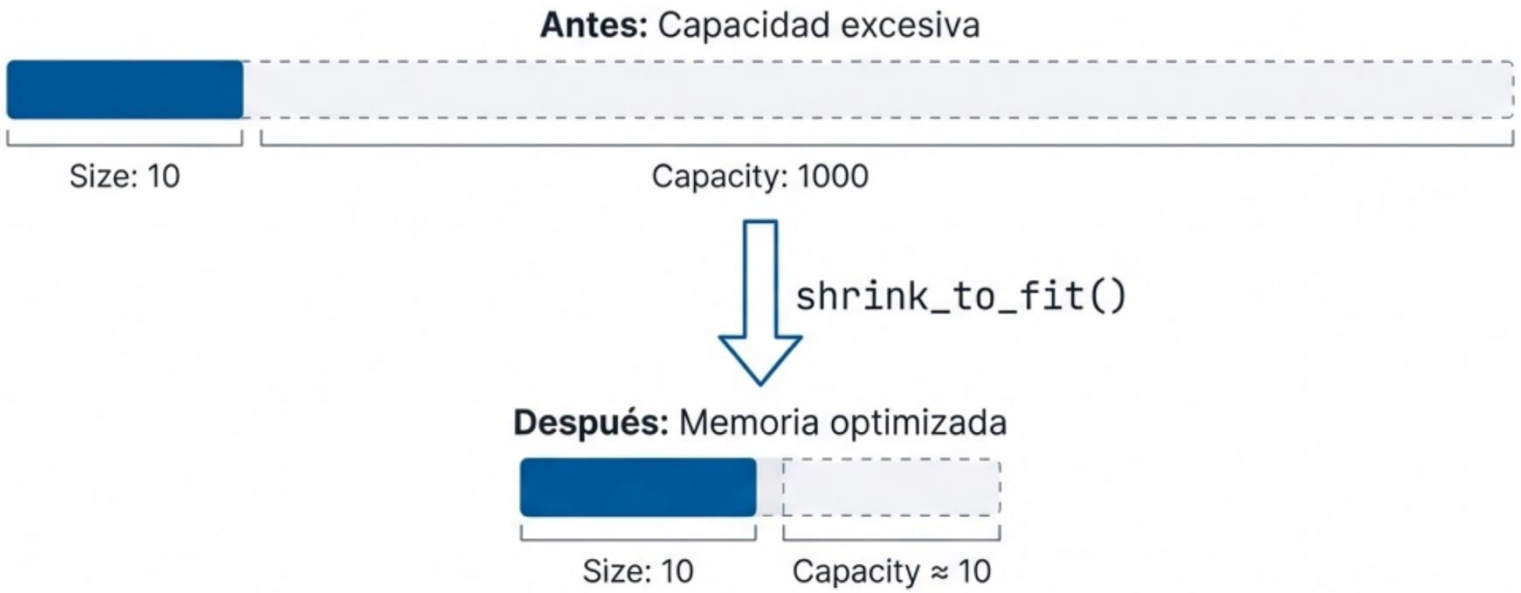
```
std::vector<int> numbers;
numbers.reserve(100000); //
Reserva memoria de una sola
vez
// Insertar elementos ahora
es mucho más rápido
for(int i=0; i<100000; ++i)
  numbers.push_back(i);
```



Evita múltiples reasignaciones y copias de memoria.

Optimización de Memoria: shrink_to_fit()

Solicitud para reducir la capacidad y liberar memoria no utilizada.




```
numbers.shrink_to_fit(); // Libera memoria no usada
```

Declaración e Inicialización (Estándar C++17)

Declaración Vacía

```
std::vector<int> vec;
```

Tamaño y Valor por Defecto

```
// 10 elementos, todos inicializados a 5
std::vector<int> vec(10, 5);
```

Lista de Inicialización (Moderno)

```
// Sintaxis preferida con llaves {}
std::vector<int> vec = {1, 2, 3, 4, 5};

// O deducción de tipos (C++17)
std::vector vec2 = {1, 2, 3};
```

Acceso a Elementos: Velocidad vs. Seguridad

Operador [ ]

```
val = vec[10];
```

- Máxima velocidad (como C puro).
- **Sin chequeo de límites**.
- Riesgo: Undefined Behavior si el índice no existe.

Método .at( )

```
val = vec.at(10);
```

- Más seguro.
- Chequeo de límites automático.
- Lanza excepción **std::out_of_range** si falla.

Tip: Usar .front() y .back() para primer y último elemento.

Modificación Dinámica: Inserción

Construcción In-Place para máxima eficiencia

```
struct Person { Person(string n, int a); };
std::vector<Person> people;

// 1. push_back: Crea temporal + Copia/Mueve
people.push_back(Person("Lisa", 30));

// 2. emplace_back: Construye directamente en memoria
people.emplace_back("Lisa", 30);
```

→ **Más Eficiente**

`emplace_back` reenvía los argumentos al constructor, evitando la creación de objetos temporales.

Modificación Dinámica: Eliminación

Eliminar último elemento

```
vec.pop_back();
```

- Reduce size, mantiene capacity.

Limpiar todo el vector

```
vec.clear();
```

- Destruye todos los elementos. Size = 0.

Borrar posición específica

```
vec.erase(vec.begin() + 2);
```

- Requiere iteradores. Mueve los elementos posteriores (O(n)).

⚠ Nota: `clear()` no libera la memoria reservada (capacity). Usar `shrink_to_fit()` si es necesario.

Buenas Prácticas: empty() vs size()

```
if (vec.size() == 0)
```

Puede ser O(n) en otros contenedores.

```
if (vec.empty())
```

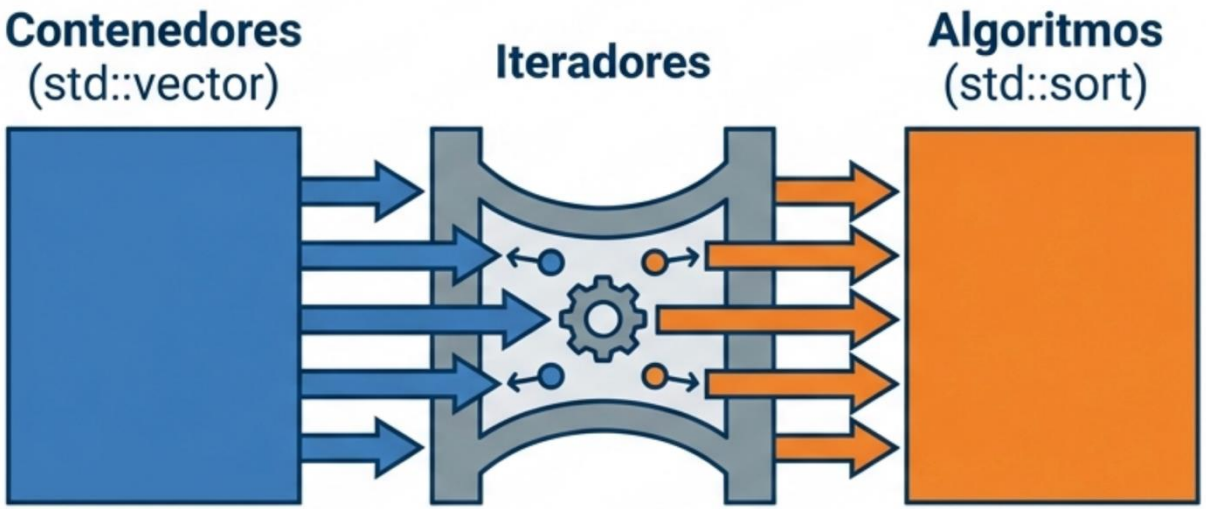
Garantizado O(1) (tiempo constante).

Semánticamente más claro y legible.

El rol del iterador

Definición: Los iteradores actúan como punteros sofisticados. Son la interfaz que conecta los contenedores (donde viven los datos) con los algortmos (la lógica que procesa los datos).

Propósito: Permiten recorrer, acceder y modificar elementos sin que el algoritmo necesite conocer la estructura interna de memoria del contenedor.

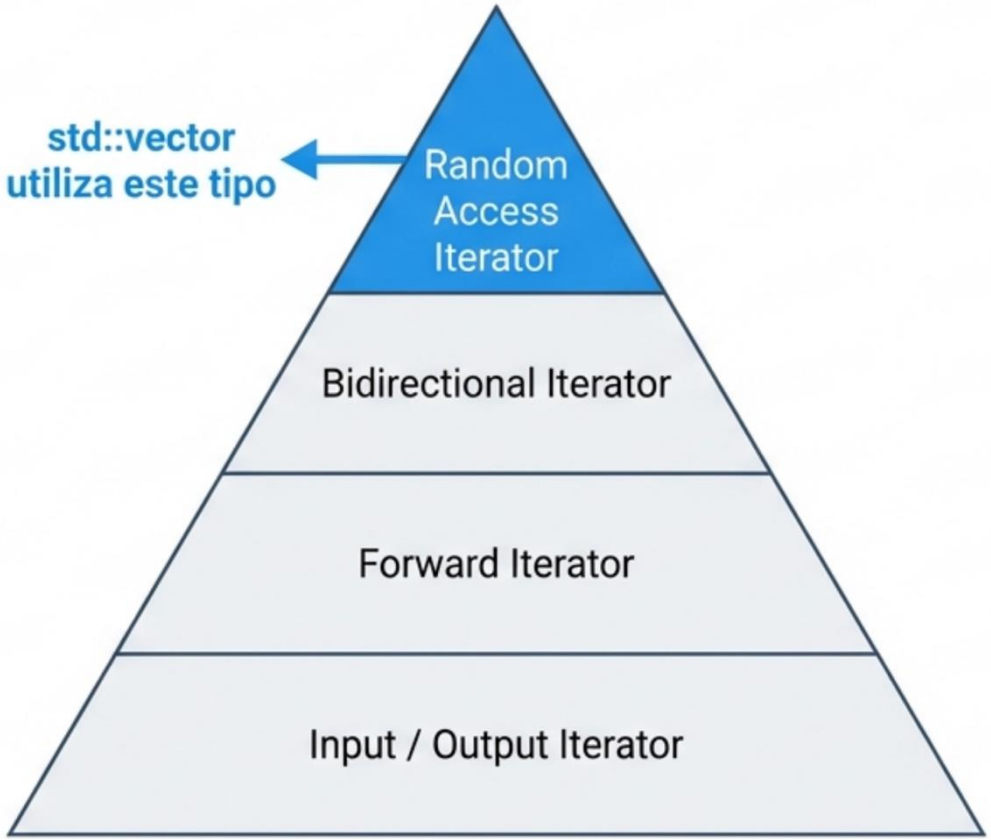



```
std::vector<int> datos = {10, 20, 30};
// El iterador conecta el vector con el algoritmo de ordenamiento
std::sort(datos.begin(), datos.end());
```

Jerarquía de Iteradores en la STL

La STL clasifica los iteradores según sus capacidades. std::vector utiliza el tipo más potente.

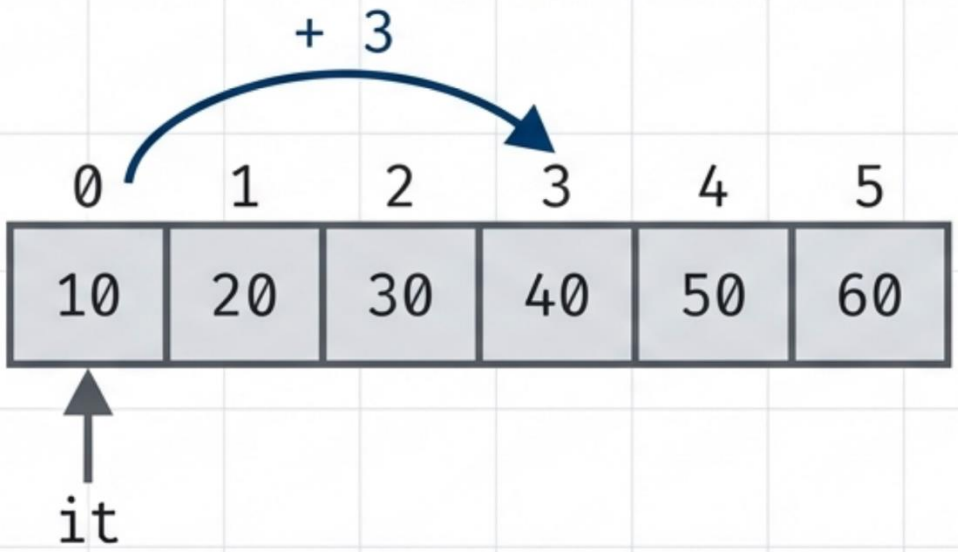
Input/Output: Lectura o escritura secuencial de un solo paso.
Forward: Lectura/escritura con desplazamiento solo hacia adelante.
Bidirectional: Desplazamiento adelante y atrás (ej. std::list).
Random Access: La capacidad más alta. Permite saltar a cualquier elemento arbitrario instantáneamente.



La Potencia de std::vector: Acceso Aleatorio

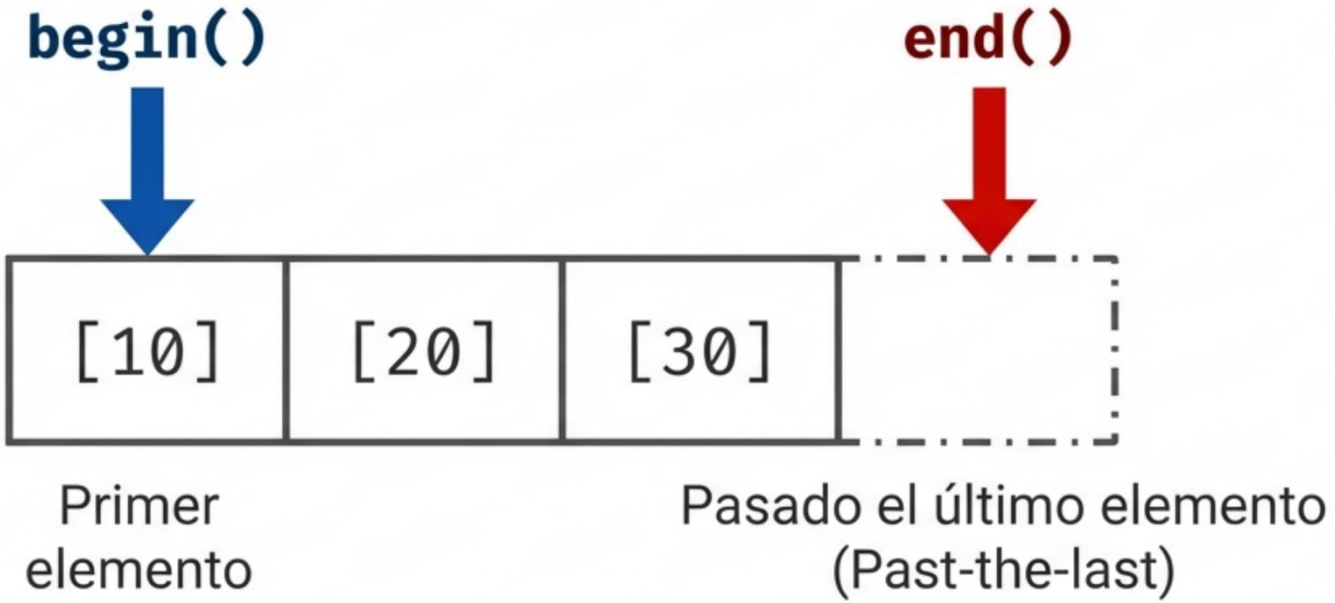
Al igual que los punteros en C, los iteradores de vectores permiten aritmética directa.

- Permite saltar a cualquier posición en tiempo constante O(1). No es necesario recorrer los elementos intermedios.
- Permite operaciones como it + 5 o it - 3.




```
std::vector<int> nums = {10, 20, 30, 40, 50};
auto it = nums.begin();

// Salto directo al 4to elemento (índice 3)
auto salto = it + 3;
std::cout << *salto; // Imprime: 40
```

Límites de Navegación: begin() y end()



Importante: Nunca se debe desreferenciar **end()**, ya que apunta a memoria fuera del rango válido.

Iteración Explícita

El patrón clásico para recorrer un vector utilizando iteradores explícitos. Aunque verbose, es fundamental entender este mecanismo.

```
std::vector<int> valores = {5, 10, 15};

// Estructura clásica del bucle for
// 1. Inicialización   2. Condición    3. Incremento
for (auto it = valores.begin(); it != valores.end(); ++it) {

    // Acceso al valor mediante desreferencia (*)
    std::cout << *it << " ";
}
// Salida: 5 10 15
```

El uso de `auto` simplifica la declaración del tipo, evitando escribir `std::vector<int>::iterator`.

Seguridad y Buenas Prácticas: Iteradores Constantes

Si la intención es solo leer datos y no modificarlos, se deben usar iteradores constantes.

- cbegin() y cend(): Garantizan que los elementos sean de solo lectura.
- Intentar modificar el valor apuntado por un iterador constante generará un error de compilación.

```
std::vector<int> valores = {100, 200, 300};

// Uso de cbegin() y cend()
for (auto it = valores.cbegin(); it !=
valores.cend(); ++it) {

   std::cout << *it << " "; // Lectura
permitida

   // *it = 500; // ERROR DE COMPILACIÓN
                  // No se puede asignar a una
ubicación de solo lectura
}
```

Funciones No Miembros: std::begin y std::end

Para escribir código más genérico y adaptable, prefiera las funciones libres definidas en <iterator>.

- **Ventaja:** Funcionan uniformemente tanto con contenedores de la STL ('std::vector') como con arrays de estilo C.

Específico:
vec.begin()
Solo funciona con contenedores STL.

Genérico:
std::begin(vec)
Funciona con STL y Arrays nativos.

```
#include <iterator>

std::vector<int> vec = {1, 2, 3};
int array_nativo[] = {4, 5, 6};

// Sintaxis unificada
auto it_vec = std::begin(vec);
auto it_arr = std::begin(array_nativo);
```

La Evolución: Bucle for Basado en Rangos

Introducido para simplificar la navegación y reducir errores errores humanos.
Internamente utiliza begin() y end() de forma automática, pero ofrece una sintaxis mucho más limpia.

Sintaxis Antigua (Verbosa)

```
for (std::vector<int>::iterator it = nums.begin();
     it != nums.end(); ++it) {
  std::cout << *it;
}
```

Sintaxis Moderna (Limpia)

```
for (int n : nums) {
  std::cout << n;
}
```

Elimina la gestión manual de iteradores y errores de límites.

Sintaxis C++17: Patrones de Bucles de Rango

Lectura por Copia

Para tipos primitivos pequeños.

```
for (auto val : nums) {
... }
```

Escritura / Modificación

Para modificar los elementos originales.

```
for (auto& val : nums) {
  val *= 2; }
```

Lectura Eficiente

Para objetos grandes (evita copias) sin modificar.

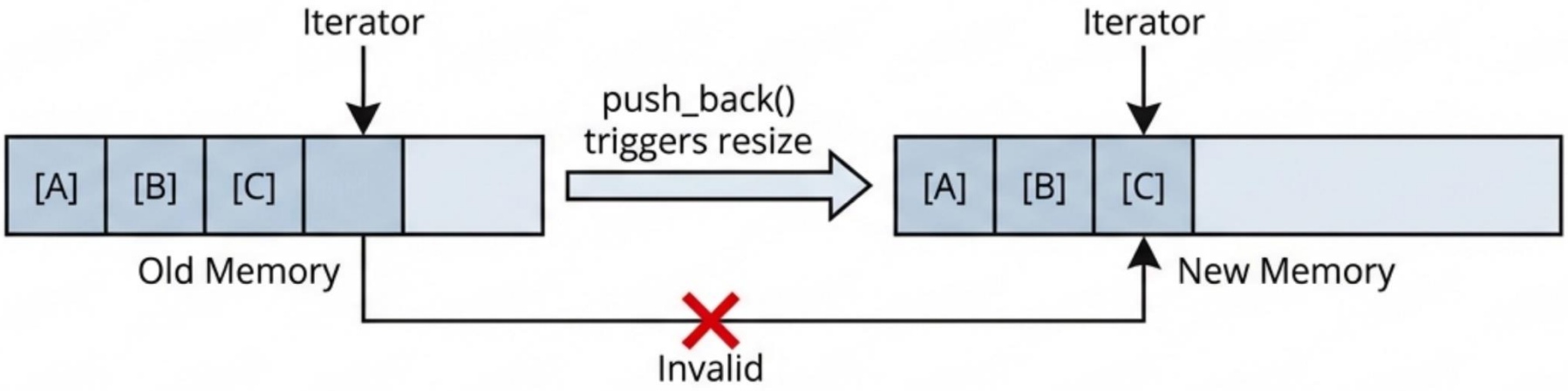
```
for (const auto& val :
nums) { ... }
```

Navegación Inversa: rbegin() y rend()

Al incrementar (++) un iterador inverso, este se mueve físicamente hacia atrás en el contenedor.

```
std::vector<char> letras = {'A',
'B', 'C'};

// Imprime: C B A
for (auto it = letras.rbegin();
it != letras.rend(); ++it) {
  std::cout << *it << " ";
}
```

Advertencia: Invalidación de Iteradores



- Operaciones que cambian el tamaño del vector (como push_back o insert) pueden causar una reasignación de memoria.
- **Riesgo:** Si el vector se mueve en memoria, todos los iteradores activos se vuelven inválidos. Usarlos causa comportamiento indefinido.
- **Consejo:** Siempre actualice los iteradores después de modificar la estructura del contenedor.

Resumen

Rendimiento

Acceso O(1) y excelente localidad de caché.

Gestión de Memoria

Automática, segura y compatible con RAII.

Flexibilidad

Crecimiento dinámico con sintaxis moderna.

Seguridad

Opciones como `.at()` para protección de límites.